

Trabalho Prático 1

Batalha de Naves – StarFleet Protocol

Disciplina de Redes de Computadores

Semestre 2025/2

1 Introdução

O ano é 3142. A humanidade já se espalhou por centenas de sistemas estelares, mas a paz interestelar não durou. Alianças foram rompidas, frotas colidem no vazio do espaço e cada batalha decide o destino de mundos inteiros.

Você é o capitão da nave **SS-42 Voyager**, uma das últimas linhas de defesa da Federação. Sua missão: enfrentar uma frota inimiga que ameaça destruir seu setor. Não há tempo para diplomacia — cada decisão de combate é tomada em segundos, e um único erro pode significar a destruição da sua nave.

A batalha é travada turno a turno. Você decide a manobra da sua nave; o inimigo reage. Os escudos brilham, torpedos atravessam o vácuo, motores de dobra se preparam para a fuga. No fim, apenas uma nave permanecerá intacta.

Bem-vindo ao **StarFleet Protocol**, onde estratégia e sorte colidem em batalhas intergalácticas.

2 Objetivos

2.1 Objetivo Geral

Desenvolver um sistema **cliente-servidor TCP** para simular batalhas entre uma nave humana (cliente) e uma nave inimiga controlada pelo computador (servidor).

2.2 Dinâmica

- O cliente (jogador) escolhe uma ação de combate.
- O servidor (inimigo) gera aleatoriamente sua ação de resposta.
- O resultado é calculado com base em uma tabela de combate.
- Cada rodada reduz a integridade das naves até que uma delas seja destruída ou o jogador decida fugir.
- Ao final da sessão, é exibido o inventário final da nave.

3 Requisitos

3.1 Requisitos Funcionais

3.2 Regras gerais

Conexão:

- O servidor deve ser inicializado com dois parâmetros: protocolo (v4 para IPv4 ou v6 para IPv6) e porta.
- O cliente conecta-se via TCP ao servidor informando endereço IP/hostname e porta.

Ações de Combate:

Opção	Ação	Efeito
0	Laser Attack	Ataque básico de energia
1	Photon Torpedo	Ataque pesado, mas pode falhar contra escudos
2	Shields Up	Defende-se de ataques de laser e torpedos
3	Cloaking	Camuflagem que evita dano, mas não ataca
4	Hyper Jump	Tentativa de fuga (encerra a partida)

Regras:

- Cada nave começa com **100 pontos de integridade (HP)**.
- Um ataque bem-sucedido causa **20 pontos de dano**.
- Shields evitam dano, Cloaking evita um ataque, Hyper Jump encerra a partida.

Feedback: Após cada turno, o servidor deve enviar:

- Ação escolhida pelo cliente.
- Ação escolhida pelo servidor.
- Resultado do turno.
- HP restante de cada nave.

Inventário Final:

- HP final da nave do cliente.
- HP final da nave inimiga.
- Total de turnos jogados.
- Número de torpedos disparados pelo cliente.
- Número de vezes que escudos foram usados.

3.2.1 Determinação do Resultado

Cada nave inicia com **100 pontos de integridade (HP)**. O dano base de um ataque bem-sucedido é de **20 HP**. O resultado de cada turno é determinado de forma *determinística* pelas regras abaixo (não há sorte além da ação aleatória escolhida pelo servidor):

Ordem de avaliação do turno

- a) **Hyper Jump tem prioridade:** se qualquer lado escolher **Hyper Jump (4)**, a partida termina imediatamente como fuga do lado que escolheu. Se ambos escolherem **Hyper Jump**, encerrar como *fuga mútua*. Nenhum dano é aplicado neste turno.
- b) **Resolução de ataques e defesas:** caso não haja **Hyper Jump**, aplicar as regras de bloqueio e de comparação de ataques abaixo.

Regras de bloqueio/evitação

- **Shields Up (2)** bloqueia completamente **Laser (0)** e **Photon Torpedo (1)**: dano = 0.
- **Cloaking (3)** evita **Photon Torpedo (1)** (sem dano), mas não evita **Laser (0)**.

Regras de confronto entre ataques

- **Photon Torpedo (1)** vence **Laser (0)**: o lado que usou **Laser** sofre 20 HP de dano.
- **Ataques idênticos:**
 - **Laser vs Laser**: ambos sofrem 20 HP.
 - **Torpedo vs Torpedo**: ambos sofrem 20 HP.

Demais combinações

- **Laser (0)** contra **Cloaking (3)**: **Laser acerta** (dano 20 no oponente).
- **Torpedo (1)** contra **Cloaking (3)**: **Cloaking evita** (dano 0).
- **Shields (2)** contra **Shields (2)** ou **Cloaking (3)**: ninguém recebe dano.

Encerramento do jogo

- Se, após a resolução do turno, $\text{client_hp} \leq 0$ ou $\text{server_hp} \leq 0$, o servidor envia **MSG_GAME_OVER** e encerra a sessão.
- Caso contrário, inicia-se um novo turno solicitando nova ação ao cliente.

Observações de implementação

- Não há limite de munição; contabilize apenas **quantas vezes** o cliente usou **Photon Torpedo** e **Shields** (inventário).
- Recomenda-se manter a lógica *determinística* para facilitar testes automatizados.

3.2.2 Feedback por Turno

Após cada turno, o servidor deve enviar:

- Ação escolhida pelo cliente.
- Ação escolhida pelo servidor.
- Resultado do turno (texto) e HP restante de cada nave.

3.2.3 Inventário Final

Ao término (destruição, fuga do cliente ou fuga do servidor), o servidor envia:

- HP final da nave do cliente.
- HP final da nave inimiga.
- Total de turnos jogados.
- Número de torpedos disparados pelo cliente.
- Número de vezes que escudos foram usados pelo cliente.

Exibir mensagem final amigável (ex.: “*Você derrotou a frota inimiga!*” / “*Sua nave foi destruída.*” / “*Você escapou para o hiperespaço.*”).

3.3 Requisitos Não Funcionais

- Linguagem: C, utilizando sockets POSIX.
- Sistema Operacional: Linux.
- Interface: terminal (sem GUI).
- Apenas um cliente por vez.

4 Comandos

4.1 Servidor

```
1 ./server v4 5001 # IPV4
2 ./server v6 5001 # IPV6
```

4.2 Cliente

```
1 ./client 127.0.0.1 5001 # IPV4
2 ./client ::1 5001 # IPV6
```

5 Protocolo de Comunicação

5.1 Estrutura da Mensagem

```
1 #define MSG_SIZE 256
2
3 typedef enum {
4     MSG_INIT,           // Boas vindas
5     MSG_ACTION_REQ,     // Solicitacao de acao
6     MSG_ACTION_RES,     // Cliente responde com acao escolhida
7     MSG_BATTLE_RESULT,  // Resultado do turno
8     MSG_INVENTORY,      // Atualizacao de inventario
9     MSG_GAME_OVER,      // Fim de jogo
10    MSG_ESCAPE           // Jogador fugiu
11 } MessageType;
12
13 typedef struct {
14     int type;
15     int client_action;
16     int server_action;
17     int client_hp;
18     int server_hp;
19     int client_torpedoes;
20     int client_shields;
21     char message[MSG_SIZE];
22 } BattleMessage;
```

6 Exemplos de Uso

Nos exemplos abaixo, por motivos de formatação deste documento, as mensagens não têm caracteres especiais. No seu código é necessário utilizar, por exemplo: você, inventário, etc.

6.1 Exemplo 1 – Vitória (Photon Torpedo acerta)

```
1 $ ./client 127.0.0.1 5000
2 Conectado ao servidor.
3 Sua nave: SS-42 Voyager (HP: 100)
4
5 Escolha sua acao:
6 0 - Laser Attack
7 1 - Photon Torpedo
8 2 - Shields Up
9 3 - Cloaking
10 4 - Hyper Jump
11
12 > 1
13 Voce disparou um Photon Torpedo!
14 Servidor usou Cloaking.
15 Resultado: Acerto! Nave inimiga perdeu 20 HP.
16 Placar: Voce 100 x 80 Inimigo
```

6.2 Exemplo 2 – Derrota (Laser contra Torpedo)

```
1 > 0
2 Voce disparou um Laser!
3 Servidor disparou um Photon Torpedo!
4 Resultado: Voce recebeu 20 de dano.
5 Placar: Voce 80 x 100 Inimigo
```

6.3 Exemplo 3 – Empate (Laser contra Laser)

```
1 > 0
2 Voce disparou um Laser!
3 Servidor disparou um Laser!
4 Resultado: Ambos receberam 20 de dano.
5 Placar: Voce 80 x 80 Inimigo
```

6.4 Exemplo 4 – Escudos bloqueiam Torpedo

```
1 > 2
2 Voce ativou os Escudos!
3 Servidor disparou um Photon Torpedo!
4 Resultado: Ataque inimigo bloqueado!
5 Placar: Voce 100 x 100 Inimigo
```

6.5 Exemplo 5 – Cloaking evita Torpedo

```
1 > 3
2 Voce ativou Cloaking!
3 Servidor disparou um Photon Torpedo!
4 Resultado: Ataque inimigo falhou!
5 Placar: Voce 100 x 100 Inimigo
```

6.6 Exemplo 6 – Cloaking nao evita Laser

```
1 > 3
2 Voce ativou Cloaking!
3 Servidor disparou um Laser!
4 Resultado: Voce recebeu 20 de dano.
5 Placar: Voce 80 x 100 Inimigo
```

6.7 Exemplo 7 – Fuga (Hyper Jump)

```
1 > 4
2 Voce acionou o Hyper Jump!
3 Sua nave escapou para o hiperespaco.
4
5 Inventario final:
6 - HP restante: 75
7 - Torpedos usados: 3
8 - Escudos usados: 2
9 Obrigado por jogar!
```

6.8 Exemplo 8 – Encerramento por destruicao do inimigo

```
1 > 1
2 Voce disparou um Photon Torpedo!
3 Servidor usou Cloaking.
4 Resultado: Acerto! Nave inimiga perdeu 20 HP.
5 Placar: Voce 100 x 0 Inimigo
6
7 Fim de jogo!
8 Inventario final:
9 - HP restante: 100
10 - Torpedos usados: 4
11 - Escudos usados: 1
12 Voce derrotou a frota inimiga!
```

6.9 Exemplo 9 – Encerramento por destruicao do cliente

```
1 > 0
2 Voce disparou um Laser!
3 Servidor disparou um Photon Torpedo!
4 Resultado: Voce recebeu 20 de dano.
5 Placar: Voce 0 x 40 Inimigo
6
7 Fim de jogo!
8 Inventario final:
9 - HP restante: 0
10 - Torpedos usados: 2
11 - Escudos usados: 3
12 Sua nave foi destruida!
```

6.10 Exemplo 10 – Entrada invalida

```
1 > 7
2 Erro: escolha invalida!
3 Por favor selecione um valor entre 0 e 4.
4
5 Escolha sua acao:
6 0 - Laser Attack
7 1 - Photon Torpedo
8 2 - Shields Up
9 3 - Cloaking
10 4 - Hyper Jump
```

7 Avaliação

- Inicialização do servidor (10%)
- Conexão cliente-servidor (15%)
- Processamento correto das ações (20%)
- Atualização correta de HP e inventário (15%)

- Encerramento por destruição (10%)
- Encerramento por fuga (5%)
- Tratamento de entradas inválidas (5%)
- Impressão correta das mensagens (5%)
- Documentação (15%)

8 Documentação

O aluno deve entregar um PDF com 3 a 6 páginas, contendo:

- Desafios encontrados.
- Estratégias adotadas.
- Possíveis melhorias.

9 Submissão

- Entrega em formato Tar gz: `TP1_MATRICULA.tar.gz`.
- A documentação somente será aceita se acompanhada de código fonte.
- Conteúdo:
 - Código-fonte em C.
 - Makefile na raiz do diretório (`$> make`, gera binários em `bin/client` e `bin/server`).
 - Documentação em PDF na raiz do diretório.
- Após a data estabelecida para entrega no Moodle não haverá possibilidade de entrega.

10 Detecção de Plágio

- O uso de código gerado por ferramentas de IA é proibido.
- Ferramentas automáticas de detecção serão utilizadas.
- Plágio resultará em nota zero.

11 Material complementar

- [Playlist de programação com sockets do professor Ítalo Cunha](#)
- [Beej's Guide to Network Programming](#)